



Escuela de Ingeniería en Computación

CURSO

Aseguramiento de la Calidad del Software

TEMA

Pruebas de Integración de Servicios API con Postman y Pytest

PROFESOR

Alicia Salazar Hernández

ESTUDIANTES

Mauricio Gonzales Prendas
Isaac Jiménez Blanco

FECHA

5 de mayo 2026

Contexto

Este laboratorio consiste en diseñar, ejecutar y documentar pruebas de integración de servicios API para un flujo funcional similar a un proceso de compra o consumo de servicios. El propósito no es únicamente validar endpoints aislados, sino comprobar que varias piezas trabajan juntas correctamente a lo largo de un flujo completo, incluyendo consulta de catálogo, selección de elementos, persistencia temporal, validación de contrato, manejo de errores y generación de métricas de calidad.

En este trabajo se combinan dos enfoques complementarios. Primero, pruebas de integración ejecutadas en Postman, incluyendo ejecución repetible con Newman. Segundo, automatización con Pytest y persistencia temporal mediante SQLite en memoria, lo cual permite validar consistencia de datos, reglas de negocio y comportamiento del sistema bajo diferentes escenarios, incluyendo casos negativos y riesgos del flujo.

Proyecto evaluado

El proyecto evaluado es Claustрум, una plataforma académica para estudiantes del Instituto Tecnológico de Costa Rica. Para efectos del laboratorio, se seleccionó un flujo centrado en la consulta de cursos y grupos disponibles y el guardado de horarios mediante Supabase REST API y funciones RPC.

El objetivo del flujo evaluado es validar que la plataforma pueda consultar el catálogo académico, obtener cursos y grupos, persistir un horario seleccionado, consultarlo posteriormente, recuperar los grupos asociados y finalmente eliminar el horario guardado, verificando tanto los contratos JSON como las reglas de negocio y el manejo de errores.

docs	feat: add API integration testing lab deliverables
evidencias	feat: add API integration testing lab deliverables
postman	feat: add API integration testing lab deliverables
scripts	feat: add API integration testing lab deliverables
tests	feat: add API integration testing lab deliverables
.env.example	feat: add API integration testing lab deliverables
.gitignore	feat: add API integration testing lab deliverables
.python-version	feat: add API integration testing lab deliverables
README.md	feat: add API integration testing lab deliverables
pyproject.toml	feat: add API integration testing lab deliverables
uv.lock	feat: add API integration testing lab deliverables

Alcance

Se probaron servicios de:

- Catálogo académico.
- Campus, carreras y planes.
- Periodos académicos.
- Cursos y grupos disponibles.
- Guardado de horarios.
- Consulta de horarios guardados.
- Consulta de grupos asociados a un horario guardado.
- Eliminación de horarios guardados.
- Manejo de errores y autenticación.

El alcance se limita a pruebas de integración a nivel API y automatización con Pytest, considerando persistencia temporal con SQLite en memoria. No se evalúan componentes internos del frontend, desempeño de infraestructura o seguridad avanzada más allá de los escenarios de autenticación y control de acceso observados en la API.

Flujo de integración elegido y adaptación del carrito

El laboratorio solicita un flujo similar a compra o carrito. En Claustum, el equivalente funcional es el “horario guardado”, ya que el usuario consulta cursos y grupos (catálogo), selecciona grupos (selección), persiste la selección (checkout) y posteriormente puede consultarla o eliminarla (carrito y limpieza del flujo).

Carrito de compras	Claustum
Productos disponibles	Cursos y grupos disponibles
Agregar al carrito	Seleccionar grupos
Consultar carrito	Consultar horario guardado
Calcular total	Calcular cursos, créditos y horas
Checkout	Guardar horario
Eliminar carrito	Eliminar horario guardado

Esta adaptación permite evaluar integración real entre componentes del catálogo, selección de entidades con IDs válidos, persistencia del horario y verificación posterior de que la información persiste y puede eliminarse correctamente.

Datos de prueba

Para asegurar repetibilidad, se utilizó un conjunto fijo de parámetros académicos durante las ejecuciones.

Campo	Valor
Universidad	1
Campus	3
Carrera	10
Plan	48
Periodo	101
Usuario	Cuenta de pruebas configurada en variables de entorno

La contraseña del usuario de pruebas no se incluye en archivos versionados ni en environments exportados. En Postman se configura como Current Value y en Newman se suministra mediante variable de entorno.

Variables de entorno

Crear un archivo local `.env` a partir de `.env.example` o exportar las variables antes de ejecutar Pytest.

```
export SUPABASE_URL="https://jxzkhfclcdpcgkpyxyk.supabase.co"
export SUPABASE_PUBLISHABLE_KEY="sb_publishable_t7JXv-4WbC00s04kruT0og_9fJ6fboa"
export CLAUSTRUM_EMAIL="pruebas@maugp.com"
export CLAUSTRUM_PASS="<password-de-pruebas>"
```

La contraseña no se incluye en archivos versionados. En Postman debe configurarse como Current Value de `claustrum_pass`.

Herramientas utilizadas

Se utilizaron las siguientes herramientas:

- Postman para ejecución manual y validación de requests.
- Newman como runner de línea de comandos para automatizar ejecuciones repetibles y generar evidencia.
- Supabase REST API y funciones RPC como servicios a integrar.
- Python con la librería requests para consumo de API.
- Pytest para automatización de pruebas de integración.
- SQLite en memoria para persistencia temporal aislada por prueba.
- uv para gestión del entorno Python y ejecución reproducible.

Estas herramientas permiten cubrir tanto validación de contratos y reglas de negocio desde Postman como automatización repetible y validación de persistencia temporal desde Pytest, cumpliendo la expectativa del laboratorio de usar herramientas reales y generar evidencia.

Matriz de casos de prueba

A continuación, se presenta la matriz de casos de prueba diseñada antes de la ejecución. El flujo evaluado corresponde a la integración de servicios API para consulta de catálogo académico y guardado de horarios en Clastrum mediante Supabase REST API y RPC.

[docs/matriz_casos_prueba.md](#) ← Se encuentra en el github

Configuración de Postman

Environment

Se utilizó un environment en Postman llamado QA_API_LAB. Este environment define variables requeridas para el flujo, incluyendo URL base de Supabase y valores dinámicos que se completan durante la ejecución (por ejemplo, IDs generados). La contraseña de pruebas no se versiona en el environment exportado, sino que se configura localmente como Current Value o mediante variables de entorno para Newman.

Colección y organización

La colección utilizada agrupa el flujo completo de integración bajo carpetas lógicas: autenticación, catálogo académico, horarios guardados y pruebas negativas. Esto permite ejecutar el flujo como una sola suite y validar continuidad entre pasos.

Validaciones por endpoint

Se incluyeron validaciones mínimas por endpoint importante, cubriendo:

- Estado HTTP esperado (200, 204, 400, 403 según el caso).
- Tiempo de respuesta bajo el umbral establecido.
- Contrato de respuesta (campos esperados) y reglas de negocio (por ejemplo existencia de datos configurados o persistencia del horario guardado).

Name	Last commit message
..	
Integracion_Clastrum_Horarios.postman_collection.json	feat: add API integration testing lab deliverables
QA_API_LAB.postman_environment.json	feat: add API integration testing lab deliverables

Evidencia y resultados de ejecución con Newman

La colección Postman fue ejecutada automáticamente usando Newman durante 3 iteraciones, con el fin de demostrar repetibilidad del flujo completo y registrar evidencia CLI. El reporte de consola incluye el detalle por request y un resumen final.

Resumen de ejecución Newman:

Iterations: 3

Requests: 45

Assertions: 135

Failed: 0

Average response time: 229 ms

Evidencia almacenada en [evidencias/newman/resultados-20260505-173726.txt](#). No se conserva un reporte JSON porque Newman puede incluir valores resueltos de variables secretas en el export.

293			
294		executed	failed
295			
296	iterations	3	0
297			
298	requests	45	0
299			
300	test-scripts	90	0
301			
302	prerequest-scripts	45	0
303			
304	assertions	135	0
305			
306	total run duration: 11.2s		
307			
308	total data received: 415.81kB (approx)		
309			
310	average response time: 229ms [min: 169ms, max: 526ms, s.d.: 73ms]		
311			

Automatización con Pytest y SQLite en memoria

Se implementó una suite automatizada en `tests/test_api_integration.py` para validar el flujo de integración desde Python. Las pruebas consumen la API de Supabase, verifican contrato y reglas del flujo, y utilizan SQLite en memoria como persistencia temporal aislada por prueba. Esto permite validar la consistencia de la selección, almacenamiento y cálculo de totales de forma controlada, sin depender de una base persistente local.

La suite cubre:

- Contrato del catálogo académico.
- Contrato y tiempo de respuesta de cursos y grupos.
- Inserción temporal en SQLite y cálculo de totales de cursos, créditos y horas.
- Guardado autenticado de horario y verificación de persistencia mediante consulta posterior.
- Eliminación del horario guardado y validación de ausencia.
- Pruebas negativas: sin autenticación, nombre vacío y grupos inválidos.
- Documentación del comportamiento actual de guardado parcial cuando el payload mezcla grupos válidos e inválidos.

Resultado de ejecución Pytest:

9 passed in 10.09s

Evidencia en [evidencias/pytest/resultados.txt](#).

```
1  ===== test session starts =====
2  platform linux -- Python 3.13.11, pytest-9.0.3, pluggy-1.6.0 -- /home/mau/Dev/tec/qa/tarea07/.venv/bin/python
3  cachedir: .pytest_cache
4  rootdir: /home/mau/Dev/tec/qa/tarea07
5  configfile: pyproject.toml
6  collecting ... collected 9 items
7
8  tests/test_api_integration.py::test_catalog_flow_contract PASSED [ 11%]
9  tests/test_api_integration.py::test_schedule_courses_contract_and_response_time PASSED [ 22%]
10 tests/test_api_integration.py::test_insert_selected_groups_in_sqlite_and_calculate_totals PASSED [ 33%]
11 tests/test_api_integration.py::test_save_schedule_and_load_group_lookups_persists_selection PASSED [ 44%]
12 tests/test_api_integration.py::test_delete_saved_schedule_removes_it PASSED [ 55%]
13 tests/test_api_integration.py::test_save_schedule_requires_authentication PASSED [ 66%]
14 tests/test_api_integration.py::test_save_schedule_rejects_empty_name PASSED [ 77%]
15 tests/test_api_integration.py::test_save_schedule_rejects_invalid_groups PASSED [ 88%]
16 tests/test_api_integration.py::test_partial_invalid_groups_are_saved_as_current_behavior PASSED [100%]
17
18  ===== 9 passed in 10.09s =====
```

Métricas obligatorias y resultados

Se definieron y reportaron las métricas mínimas requeridas por el laboratorio, utilizando resultados de Newman y Pytest como evidencia.

Métrica	Cálculo	Criterio	Resultado
Porcentaje de casos aprobados	Casos aprobados / total x 100	$\geq 85\%$	100% en Pytest: 9/9
Tiempo de respuesta	ms por endpoint crítico	< 5000 ms	Cumplido en ejecuciones; max observado 526 ms
Cobertura de endpoints	Endpoints probados / endpoints del flujo x 100	$\geq 80\%$	Flujo cubierto en catálogo + guardado + consulta + eliminación
Validaciones por endpoint	Assertions / requests	≥ 3 por endpoint importante	135 / 45 = 3.0
Tasa de error controlado	Negativos controlados / negativos ejecutados	100%	Negativos controlados: sin auth, nombre vacío, grupo inexistente
Defectos encontrados	Hallazgos funcionales o técnicos	Registrar severidad	Riesgo de guardado parcial documentado (severidad media)

Defectos, hallazgos y riesgos

Durante la ejecución y análisis del flujo se documentaron los siguientes hallazgos:

Severidad	Hallazgo	Impacto	Evidencia	Recomendación
Media	save_user_schedule permite guardado parcial si el payload mezcla grupos válidos e inválidos	El usuario puede creer que guardó todo el horario cuando solo se persistió una parte	Caso automatizado de comportamiento actual	Rechazar toda la solicitud si algún grupo no existe o devolver detalle explícito de grupos omitidos
Baja	La carga de horarios depende de que los grupos sigan disponibles en el filtro actual	Un horario antiguo podría cargarse incompleto si cambian los datos activos	Revisión del flujo de carga en Claustum	Informar grupos faltantes y comparar cantidad esperada vs recuperada

Conclusiones

- Una prueba de API aislada valida un endpoint individual, mientras que una prueba de integración valida la continuidad del flujo completo entre catálogo, selección, persistencia, consulta y eliminación, confirmando que las dependencias entre servicios funcionan correctamente.
- Probar únicamente la interfaz gráfica podría ocultar defectos como errores de contrato JSON, políticas de acceso (RLS), permisos, persistencia parcial y manejo de payloads inválidos, ya que estos problemas pueden no ser evidentes desde la UI hasta que el usuario enfrenta inconsistencias.
- Las validaciones más importantes en este flujo fueron contrato JSON, reglas de negocio y persistencia, debido a que el proceso depende de IDs correctos, relaciones entre tablas y consistencia al guardar y recuperar horarios.
- Integrar servicios externos sin pruebas automatizadas incrementa el riesgo de cambios de contrato, fallos de autenticación y errores de datos no detectados, lo que puede romper flujos críticos sin advertencia temprana.

- Para un escenario de producción se recomienda integrar la suite automatizada en un pipeline de CI, usar datos semilla controlados y endurecer validaciones en el RPC save_user_schedule para evitar guardados parciales o informar explícitamente resultados parciales al cliente.

Links importantes:

<https://github.com/mau671/QA-Tarea07> ← Link del Github

<https://claustrum.maugp.com/> ← Link de la app web